Vulnerability Assessment Report

Read-Only, Passive-Scope Security Review

Target: http://testphp.vulnweb.com

Prepared by: Maxwel | Future Interns — Cyber Security Task 1 (2026)

Assessment Date: June 2026

Confidentiality & Scope Statement

This report is the result of a read-only, passive security review performed against a publicly designated test target. No login bypass, exploitation, brute-force, or denial-of-service activity was attempted at any point. All checks were limited to publicly available pages, response headers, and browser-visible configuration, consistent with an ethical, non-disruptive review. Findings reflect a point-in-time snapshot and should be re-verified if the target environment changes.

1. Executive Summary

This assessment reviewed the public-facing configuration of http://testphp.vulnweb.com to identify common, low-risk-to-test security weaknesses that a typical small business website might also carry — things like missing encryption, missing protective headers, and outdated software signals. No attempt was made to access restricted areas, submit malicious input, or exploit any vulnerability; the goal was to assess exposure the way an outside visitor or automated scanner would see it.

Seven findings were identified: two rated High risk, four rated Medium risk, and one rated Low risk. The most pressing issue is the complete absence of HTTPS encryption across the site, including on the login form, which exposes visitor data — and potentially credentials — to interception. Most of the remaining issues are configuration gaps that are inexpensive to close and would meaningfully raise the site's baseline security posture.

Risk Level	Count	Examples
High	2	No HTTPS, login over HTTP
Medium	4	Missing headers, HSTS, cookies, old stack
Low	1	Server version disclosure

2. Scope & Methodology

In Scope

- Public-facing pages of the target only
- Passive observation of HTTP response headers
- Browser DevTools inspection (cookies, network responses, page source)
- TLS/HTTPS configuration check

Out of Scope (Not Performed)

- Login bypass or credential testing
- Exploitation of any kind (e.g., injection, file upload abuse)
- Brute-force or denial-of-service activity
- Any action that could disrupt or alter the live site

Tools Used

- SecurityHeaders.com — automated header analysis
- SSL Labs (Qualys) — TLS/HTTPS configuration check
- Browser DevTools — manual inspection of headers, cookies, and page source

3. Summary of Findings

ID	Finding	Risk
F-01	Site Served Without Encryption (No HTTPS)	**High**
F-02	Login Form Reachable Over Plain HTTP	**High**
F-03	Missing Recommended Security Headers	**Medium**
F-04	Missing HTTP Strict Transport Security (HSTS)	**Medium**
F-05	Indications of an Outdated Backend Technology Stack	**Medium**
F-07	Cookies Missing Secure / HttpOnly Attributes	**Medium**
F-06	Server Software Version Disclosed in Responses	**Low**

4. Detailed Findings

F-01 — Site Served Without Encryption (No HTTPS)

Risk Level	High
What we found	The website is reachable over plain “http://” rather than an encrypted “https://” connection. There is no valid TLS certificate enforcing secure transport across the site.
Why it matters	Any information sent between a visitor and the server — page requests, form data, session identifiers — travels as plain, readable text. Anyone positioned on the same network path (public Wi-Fi, a compromised router, an ISP) can read or tamper with that traffic. Modern browsers also flag HTTP sites as “Not Secure,” which damages visitor trust.
Recommended fix	Install a TLS certificate (a free option such as Let’s Encrypt works well), configure the web server to redirect all HTTP requests to HTTPS, and confirm the redirect with a quick browser test.

F-02 — Login Form Reachable Over Plain HTTP

Risk Level	High
What we found	The site’s login functionality is accessible without being forced onto an encrypted connection, meaning a username and password could be typed and submitted over HTTP.
Why it matters	Login credentials are some of the most sensitive data a site handles. If submitted over an unencrypted connection, they can be captured in transit, leading directly to account takeover.
Recommended fix	Once HTTPS is enabled site-wide (see F-01), explicitly block authentication pages from loading over HTTP and ensure forms post only to HTTPS endpoints.

F-03 — Missing Recommended Security Headers

Risk Level	Medium
What we found	Standard protective HTTP response headers — Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, and Referrer-Policy — are not present in the server's responses.
Why it matters	These headers tell a visitor's browser how to behave safely. Without them the site is more exposed to clickjacking (tricking a user into clicking something invisible), browsers misreading file types in unsafe ways, and malicious scripts being more damaging if one is ever injected.
Recommended fix	Add the missing headers at the web server or CDN level: Content-Security-

	Policy (start with a reporting-only policy), X-Frame-Options: SAMEORIGIN, X-Content-Type-Options: nosniff, and Referrer-Policy: strict-origin-when-cross-origin. Most servers (Nginx, Apache, Cloudflare) support this with a few configuration lines.

F-04 — Missing HTTP Strict Transport Security (HSTS)

Risk Level	**Medium**
What we found	The site does not send a Strict-Transport-Security header, which is the mechanism that tells a browser to always use HTTPS for this domain in future visits.
Why it matters	Without HSTS, a visitor's very first connection (or any connection after clearing cookies) can still be silently downgraded to HTTP by an attacker, even if HTTPS is technically available.
Recommended fix	After HTTPS is stable and confirmed working sitewide, add a Strict-Transport-Security header starting with a short max-age, then increase it once confidence in the HTTPS setup is established.

F-05 — Indications of an Outdated Backend Technology Stack

Risk Level	**Medium**
What we found	Page behaviour and visible source details suggest the site runs on an older version of its underlying PHP-based framework rather than a current, actively maintained release.
Why it matters	Software that is no longer on a supported version stops receiving security patches. Publicly known weaknesses in older versions remain permanently open, since no fix is ever issued for that version.
Recommended fix	Inventory the current framework/language version, plan an upgrade to a supported release, and put a recurring patch-review schedule in place (e.g., monthly) so the stack doesn’t fall behind again.

F-06 — Server Software Version Disclosed in Responses

Risk Level	**Low**
What we found	The server identifies its software name and version in its response headers rather than keeping that detail private.
Why it matters	This isn’t harmful by itself, but it gives anyone probing the site a head start: they can look up known weaknesses for that exact version instead of guessing.
Recommended fix	Configure the web server to suppress or generalize its version banner (e.g., disable “Server” and “X-Powered-By” details in the server configuration).

F-07 — Cookies Missing Secure / HttpOnly Attributes

Risk Level	**Medium**
What we found	Cookies set by the site do not appear to include the Secure and HttpOnly attributes when inspected in the browser.
Why it matters	Without “Secure,” a cookie can be sent over an unencrypted connection. Without “HttpOnly,” a cookie can be read by a malicious script running on the page. Either gap increases the chance a session could be hijacked.
Recommended fix	Set Secure, HttpOnly, and SameSite=Lax (or Strict, where workflow allows) on every cookie, especially any cookie used to track a logged-in session.

5. Remediation Roadmap

Suggested order of work, prioritizing the highest-impact, lowest-effort fixes first:

Priority	Action	Findings Addressed
1	Enable HTTPS site-wide and redirect all HTTP traffic	F-01, F-02
2	Add HSTS header once HTTPS is confirmed stable	F-04
3	Add core security headers (CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy)	F-03
4	Set Secure, HttpOnly, and SameSite on all cookies	F-07
5	Plan and schedule a backend framework/version upgrade	F-05
6	Suppress server version banners in web server config	F-06

6. Conclusion

Overall, the target site shows the kind of foundational gaps that are common on sites that haven’t had a recent security review: no enforced encryption, missing protective headers, and signs of an aging backend. None of these require advanced technical skill to fix — most are configuration changes a developer or hosting provider can apply directly. Addressing the two High-risk items (HTTPS and the unencrypted login path) should be treated as the immediate priority, with the remaining items rolled out over the following weeks as part of routine maintenance.

7. Appendix: Supporting Evidence

The following evidence captures should be inserted here to accompany this report:

- Screenshot 1 — SecurityHeaders.com scan result for http://testphp.vulnweb.com
- Screenshot 2 — SSL Labs (Qualys) scan attempt for testphp.vulnweb.com
- Screenshot 3 — Browser DevTools → Network tab → response headers for the homepage request
- Screenshot 4 — Browser DevTools → Application tab → Cookies, showing missing Secure/HttpOnly flags